Oracle WebLogic 14c for C2M — Install, Configure & Domain Setup

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

The problem . . . 4

The solution . . . 4

What's included . . . 4

Business value . . . 5

Bottom line . . . 5

Solution Overview 5

The named topology this solution builds . . . 5

Domain topology — how C2M sits on WebLogic . . . 6

The deployment model — Assess → Configure → Pilot → Validate → Expand . . . 7

What's inside this solution . . . 7

Outcome . . . 8

Requirements Guide 8

1. Certified WebLogic 14c install media + a certified JDK . . . 8

2. Host meeting WebLogic 14c + C2M sizing (CPU, memory, disk, OS) . . . 9

3. OS product account + directory layout (`ORACLE_HOME`, `DOMAIN_HOME`) . . . 9

4. Network reachability + planned ports (Admin, managed, Node Manager) . . . 9

5. C2M database reachable + a schema/service to point the data source at . . . 10

6. WebLogic admin credentials + a boot-identity plan . . . 10

7. Administrative tooling: WLST, shell, SQL client . . . 10

8. Change control, backup & a non-production pilot . . . 11

Global Requirements — Oracle Utilities C2M / WebLogic Solutions 11

1. Licensed Oracle Utilities C2M + WebLogic entitlement . . . 12

2. Certified platform stack (OS, Java, Database, WebLogic) . . . 12

3. Environment topology (Dev / Test / Prod) & access . . . 12

4. Administrative access & tooling (WLST, shell, SQL) . . . 12

5. Change control, backup & a non-production pilot . . . 13

6. Collaborating teams & support model . . . 13

Implementation Guide 13

Phase 0 — Assess (gate before you build) . . . 14

Phase 1 — Configure . . . 14

Phase 2 — Pilot (one non-production node) . . . 18

Phase 3 — Validate (evidence, not assumption) . . . 18

Phase 4 — Expand . . . 18

Rollback (any phase) . . . 19

Testing Guide 19
1. Installation & domain structure . . . 19
2. Node Manager running and reachable . . . 19
3. Administration Server RUNNING . . . 20
4. Managed server(s) RUNNING . . . 20
5. Data source healthy (Test-On-Reserve) . . . 20
6. Database round-trip from the host (independent of WebLogic) . . . 20
7. JVM / startup configuration in effect . . . 21
8. Coverage / regression — the Report . . . 21
Test summary gate . . . 21
Troubleshooting Guide 22
Install & domain creation . . . 22
Administration Server . . . 23
Node Manager . . . 23
Managed servers . . . 24
Data sources (`C2MDataSource`) . . . 24
JVM / startup / memory . . . 25
When to escalate . . . 25
Best Practices 25
Domain layout & topology . . . 26
Node Manager . . . 26
Data sources . . . 26
JVM / startup & memory . . . 27
Security & operations . . . 27
Validate continuously . . . 28
Deliverables Summary 28
Documentation . . . 28
Automation scripts . . . 29
How the pieces work together . . . 30
What this solution does not include (pointed to, not reproduced) . . . 30
Outcome . . . 30
Appendix A — PowerShell Scripts 31

Executive Summary

Oracle Utilities **Customer to Meter (C2M)** runs on **Oracle WebLogic Server** — and a mis-built WebLogic domain undermines everything above it. Sloppy domain topology, no Node Manager, JVMs sized by guesswork, or hand-typed database connections produce a middleware layer that looks fine in a demo and fails under real load. This solution builds that layer correctly: **install and configure Oracle WebLogic 14c (14.1.1) as the C2M foundation** — the domain, Administration and managed servers, Node Manager, data sources, and JVM / startup configuration — **pilot-first**, with explicit gates.

The problem

The middleware foundation is where C2M problems start but are hardest to see. A domain built without discipline leaves predictable failures downstream: servers that won't stay up, connection-pool errors under concurrency, no automated restart when a server crashes, and memory exhaustion when the JVM is left at template defaults. Because C2M, batch, integrations, and monitoring all sit **on top** of the domain, a weak foundation multiplies into every layer above it — and re-doing the base after the fact is expensive and disruptive.

The solution

A self-guided, tested implementation that builds the WebLogic 14c foundation the way an experienced middleware administrator would, following **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with pass/fail gates and a documented rollback, always on a **non-production environment first**:

- **Install WebLogic 14c** into a clean `ORACLE_HOME` on a certified stack.
- **Create the** `c2m_domain` **domain** with a clear, named topology — `AdminServer`, managed server(s), cluster, and machine — outside the binaries so patching never disturbs it.
- **Configure the per-domain Node Manager** so servers start, stop, and **auto-restart** under control, and start unattended via a boot identity.
- **Create the** `C2MDataSource` connection pool to the C2M database with **Test-On-Reserve** health checking, targeted at the managed server(s) — not the Admin Server.
- **Tune JVM / startup** deliberately (heap, metaspace, production mode) for C2M's memory profile.
- **Validate** the whole foundation with evidence — servers `RUNNING`, Node Manager reachable, data sources healthy, JVM args in effect — before scaling out.

What's included

Full documentation — Solution Overview, Requirements, Implementation, Testing, Troubleshooting, Best Practices, and this summary — plus **five automation scripts** (WLST/Jython, shell, SQL): read-only **Discovery**, **Validation**, and **Report**; a **Remediation** helper that is **dry-run by default**; and a **ScopeHelper** that pins per-environment targets. Every step names the same objects and reuses them; every version-specific detail is **cited or flagged "verify for your version,"** and My-Oracle-Support-gated content is pointed to, not reproduced.

Business value

• **A foundation that holds** — a correctly built WebLogic 14c domain that every C2M layer above can rely on, proven on a pilot before it carries production load.

• **Risk reduced** — certified stack, backups, and change control; safe by construction (read-only discovery/validation, dry-run remediation) with a documented rollback at every gate.

• **Repeatable across environments** — the same scripts and named topology build Dev → Test → Prod identically, so the domain that reaches production is the one you validated.

• **A fraction of consulting cost** — the middleware expertise of a WebLogic administrator, packaged and self-guided.

Bottom line

A correctly built **WebLogic 14c foundation for C2M** — domain, servers, Node Manager, and data sources healthy — **proven on a pilot before you scale.**

Solution Overview

Oracle Utilities **Customer to Meter (C2M)** — and the Oracle Utilities Application Framework (OUAF) beneath it — is a Java EE application that runs on **Oracle WebLogic Server**. Everything above the middleware (online, web services, batch threadpools) depends on a WebLogic **domain** that is built and configured correctly. A domain with a sloppy topology, no Node Manager, JVMs sized by guesswork, or hand-typed connection strings for data sources will haunt every layer that sits on top of it.

This solution installs and configures **Oracle WebLogic Server 14c (14.1.1)** as the C2M middleware foundation: the **domain**, the **Administration Server** and **managed servers**, the **Node Manager** that controls their lifecycle, the **JDBC data sources** to the C2M database, and the **JVM / startup** configuration OUAF expects — built pilot-first and validated before it carries production load.

> ***Scope boundary.*** *This solution builds and configures the **WebLogic domain** that C2M is deployed onto (install WebLogic → create domain → servers → Node Manager → data sources → JVM / startup → validate). Deploying the C2M/OUAF EAR (`SPLWeb`, `SPLService`) onto the domain, batch threadpool workers, Coherence/JMS/IWS internals, SSO/SSL, and patching each have their own solutions and are pointed to, not reproduced here. Where a C2M release pins a specific WebLogic / JDK combination, that is a **"verify for your version"** check against Oracle's certification matrix (see the Requirements Guide) — never assumed.*

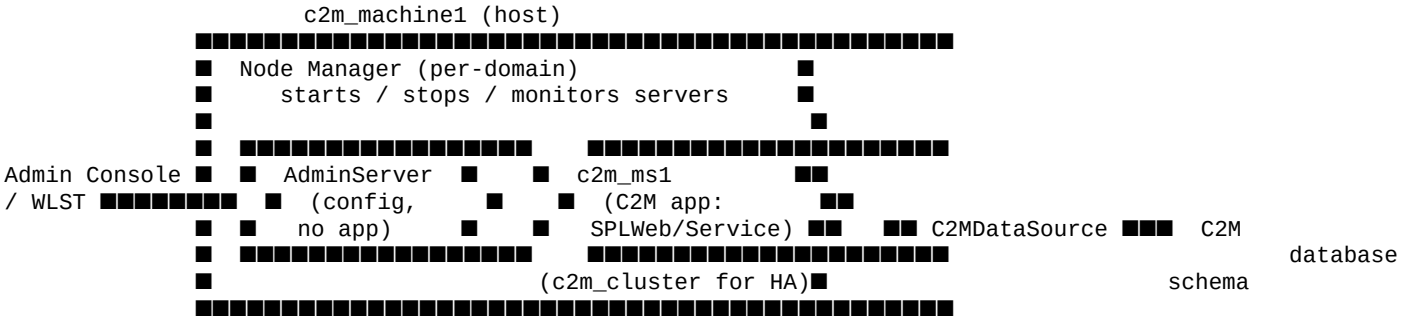
The named topology this solution builds

Every step, script, and check in this solution refers to the **same named objects**. Adopt these names (or map them once to your site's convention and reuse yours consistently) so the domain, scripts, and runbooks all speak the same language:

Object	Name used throughout	Role
WebLogic domain	`c2m_domain`	The domain OUAF/C2M is deployed onto
Domain home	`DOMAIN_HOME` = `.../c2m_domain`	Config, `bin/` startup scripts, servers' state
Administration Server	`AdminServer`	Config authority + Admin Console / WLST target
Managed server(s)	`c2m_ms1` (and `c2m_ms2` for HA)	Where the C2M application runs
Cluster (HA topology)	`c2m_cluster`	Groups the managed servers for HA/scale
Machine	`c2m_machine1`	Ties servers to a host + its Node Manager
Node Manager	per-domain, in `DOMAIN_HOME/nodemanager`	Starts/stops/monitors servers
JDBC data source	`C2MDataSource`	Connection pool to the C2M database schema
OS product account	the OUAF product user (e.g. `cissys`)	Owns the WebLogic + OUAF file system

`ORACLE_HOME` is the WebLogic 14c install (the Middleware / Fusion Middleware home); `DOMAIN_HOME` is the domain, placed **outside** `ORACLE_HOME` per Oracle's guidance so an upgrade or reinstall of the binaries does not disturb the domain.

Domain topology — how C2M sits on WebLogic



- **Administration Server (`AdminServer`)** — the single point of configuration truth for the domain. You administer it through the **WebLogic Administration Console** and **WLST**. Best practice: **do not deploy the C2M application to the AdminServer** — keep it for administration and run C2M on the managed server(s).
- **Managed server(s) (`c2m_ms1`, optionally `c2m_ms2`)** — where the C2M/OUAF application (`SPLWeb` online + `SPLService` web services) actually runs and serves users and integrations. A single managed server is a valid pilot; two-in-a-cluster (`c2m_cluster`) is the usual production HA shape.

• **Node Manager** — the WebLogic utility that **starts, stops, restarts and monitors** the Admin

and managed servers, including auto-restart of a crashed server. This solution uses the **per-domain** Node Manager (config lives under `DOMAIN_HOME/nodemanager`), which is the modern default and keeps each domain self-contained; the alternative **per-host** Node Manager is noted where it matters. Servers are enrolled with the `nmEnroll` WLST command so Node Manager can manage them.

• **Machine (`c2m_machine1`)** — the WebLogic representation of a host. Servers are assigned to a

machine, and the machine points at that host's Node Manager listen address (a **specific IP / hostname, not** `localhost`), which is how the Admin Console/WLST reach Node Manager to control servers.

• **JDBC data source (`C2MDataSource`)** — the WebLogic-managed connection pool to the C2M

database schema. OUAF connects to the database **through** this data source, so its driver, URL, credentials, pool sizing, and **Test-On-Reserve** health check are domain configuration, not application code. On an Oracle RAC database, this is typically an **Active GridLink** data source.

• **JVM / startup configuration** — heap and GC flags, `USER_MEM_ARGS` / `setDomainEnv`

customization, production mode, and a boot-identity file so servers start unattended under Node Manager. C2M is memory- and GC-sensitive; this is set deliberately, not left at template defaults.

The deployment model — Assess → Configure → Pilot → Validate → Expand

Consistent with every Oracle-category solution, the domain is built with explicit gates and a documented rollback, always on a **non-production environment first**:

1. **Assess** — confirm entitlement, a **certified** OS/JDK/DB/WebLogic combination for the target

C2M release, host resources, DB connectivity, and access (OS product account, WebLogic admin, schema). *Gate: certified stack + reachable database, or stop.* (See Requirements Guide.)

1. **Configure** — install WebLogic 14c, create `c2m_domain`, configure `AdminServer` + managed

server(s) + `c2m_machine1`, stand up the per-domain **Node Manager**, create `C2MDataSource`, and set JVM / startup. *Gate: config valid, no template placeholders left in.*

1. **Pilot** — bring the domain up on **one** non-production environment / node: Node Manager

running, `AdminServer` and `c2m_ms1` in `RUNNING`, `C2MDataSource` test **passes**. *Gate: a clean single-node domain before any horizontal scale.*

1. **Validate** — run the Validation and Report scripts: servers healthy, data source reserving

connections, Node Manager reachable, JVM args as intended, logs clean. *Gate: evidence, not assumption.*

1. **Expand** — add the second managed server / cluster, additional machines/nodes, and promote the

pattern across environments (Dev → Test → Prod) under change control, re-validating at each step.

What's inside this solution

• **Requirements Guide** — each prerequisite (entitlement, certified stack, host sizing, DB

connectivity, access & tooling, change control) with how to **verify** and **implement**.

- **Implementation Guide** — the exact, ordered steps: install WebLogic 14c → create `c2m_domain` → configure Admin/managed servers + Node Manager + machine → create `C2MDataSource` → tune JVM/startup → validate the domain, with Admin Console navigation and WLST alongside.
- **5 automation scripts (WLST/Jython, shell, SQL)** — read-only **Discovery**, read-only **Validation**, a read-only **Report**, a **Remediation** helper (dry-run by default), and a **ScopeHelper** that pins the domain/server/data-source targets the other scripts act on.
- **Testing, Troubleshooting, and Best Practices** guides — prove the domain is up, managed servers running, and data sources healthy; diagnose the common failures; and lay the domain out for the long run.

Outcome

A correctly built **WebLogic 14c foundation for C2M** — `c2m_domain` created, `AdminServer` and the managed server(s) `RUNNING`, the per-domain **Node Manager** in control, `C2MDataSource` healthy, and JVM / startup configured deliberately — **proven on a pilot before you scale**, so every layer C2M adds on top starts from solid ground.

Requirements Guide

These requirements are **specific to installing and configuring WebLogic 14c as the C2M middleware foundation**. They are in addition to the Oracle-category **Global Requirements** (licensed C2M/WebLogic entitlement, certified platform stack, Dev/Test/Prod topology, administrative access & tooling, change control/backup/pilot, collaborating teams) — verify those first, then these.

Each requirement below states **Description**, **Why**, how to **Verify**, and how to **Implement if missing**. Where a value depends on the release, it is flagged **verify for your version** against Oracle's certification matrix on My Oracle Support (MOS) — never assumed.

1. Certified WebLogic 14c install media + a certified JDK

- **Description:** The **Oracle WebLogic Server 14c (14.1.1)** installer (the generic `.jar` / Fusion Middleware Infrastructure installer, per what the C2M release requires) and a **certified JDK** (a supported Oracle JDK / Java SE version) for that WebLogic + C2M combination.
- **Why required:** WebLogic 14c installs into an `ORACLE_HOME` using a JDK; C2M/OUAF is only supported on **certified** WebLogic ↔ JDK combinations, and an uncertified JDK causes install and runtime failures and loses supportability.
- **How to verify:** Confirm the WebLogic 14.1.1 installer is downloaded from Oracle (eDelivery / MOS) and its checksum matches; run `java -version` and confirm the JDK version and vendor are on the C2M release's certification matrix. **Verify for your version** — the exact WebLogic 14.1.1.x patch level and JDK build are pinned per C2M release.
- **How to implement if missing:** Obtain the certified installer and JDK via your entitlement;

install the JDK first and set `JAVA_HOME` before installing WebLogic.

2. Host meeting WebLogic 14c + C2M sizing (CPU, memory, disk, OS)

• **Description:** A host (physical/VM/container) running a **certified OS** with enough **CPU,

RAM, and disk** for the WebLogic 14c binaries, the `c2m_domain` domain home, JVM heaps for the Admin + managed server(s), and OUAF product/log/temp space.

• **Why required:** C2M managed-server JVMs are memory-hungry; under-provisioned memory or disk

causes `OutOfMemoryError`, failed domain creation, or servers that will not stay `RUNNING`.

• **How to verify:** Check OS version against the certification matrix; confirm free memory covers

the planned heaps for `AdminServer` + `c2m_ms1` (+ `c2m_ms2`) with headroom, and disk for `ORACLE_HOME`, `DOMAIN_HOME`, and logs. Confirm required OS packages/kernel settings and file-descriptor / process **ulimits** for the product user.

• **How to implement if missing:** Right-size the host, raise ulimits (e.g. `nofile`), and add

disk before building the domain. Heap sizing is **verify for your version** against C2M sizing guidance for your user/batch load.

3. OS product account + directory layout (`ORACLE_HOME`, `DOMAIN_HOME`)

• **Description:** A dedicated **OS product account** (the OUAF product user, e.g. `cissys`) that

owns the WebLogic install and the domain, with a planned layout: `ORACLE_HOME` for the WebLogic 14c binaries and a **separate** `DOMAIN_HOME` for `c2m_domain` **outside** `ORACLE_HOME`.

• **Why required:** Running WebLogic and OUAF under a single, correctly-permissioned product

account is required for supportability; keeping the domain outside the binaries lets you patch or reinstall WebLogic without disturbing `c2m_domain`.

• **How to verify:** Confirm the product user exists, owns (or can create) both paths, and that the

paths are on adequately-sized filesystems. Confirm the account's shell profile sets `JAVA_HOME` and can reach WebLogic's `common/bin`.

• **How to implement if missing:** Create the product user/group, provision the two directory trees

with correct ownership, and place the domain home outside `ORACLE_HOME`.

4. Network reachability + planned ports (Admin, managed, Node Manager)

• **Description:** Reachable, **non-conflicting** listen ports for the `AdminServer` (e.g. 7001),

each managed server, the **Node Manager**, and (if used) SSL ports and the Coherence unicast port — plus DNS/hostnames the servers and machine bind to.

• **Why required:** WebLogic servers and Node Manager bind to specific host:port pairs; a port in

use by another service, or a machine pointed at `localhost` instead of a real IP/hostname, breaks startup, Node Manager control, and (later) clustering/Coherence.

• **How to verify:** Confirm each planned port is **free** on the host (e.g. `ss -tlnp` /

`netstat`), that hostnames resolve, and that firewall rules allow admin traffic and inter-server traffic. Confirm the machine will use a **specific IP/hostname**, not `localhost`.

• **How to implement if missing:** Reserve free ports, add DNS/hosts entries, open firewall rules, and record the port map before creating the domain (the ScopeHelper script captures it).

5. C2M database reachable + a schema/service to point the data source at

• **Description:** A running **Oracle Database** hosting the C2M schema, reachable from the WebLogic host, with the connection coordinates the `C2MDataSource` will use: host(s)/SCAN, port, **service name** (not SID for RAC), and the schema credentials.

• **Why required:** `C2MDataSource` is the connection pool OUAF uses to reach C2M's data; if the database is unreachable or the service/credentials are wrong, the data source's **Test-On- Reserve** fails and C2M cannot start clean.

• **How to verify:** From the WebLogic host, confirm TCP reachability to the DB listener and a successful login to the C2M service (e.g. `sqlplus user@//host:port/service`, or `tnsping`). Confirm whether the target is single-instance or **RAC** (drives GridLink vs generic data source).

• **How to implement if missing:** Obtain the correct service name and schema credentials from the DBA, open network/firewall paths, and confirm the JDBC driver version is certified for that DB release (**verify for your version**).

6. WebLogic admin credentials + a boot-identity plan

• **Description:** The **WebLogic administrator** username/password to be set at domain creation, the plan for the **domain in production mode**, and a **boot-identity file** (`boot.properties`) so the Admin and managed servers can start unattended under Node Manager.

• **Why required:** Production-mode domains prompt for credentials at startup; without a boot identity, Node Manager cannot start servers unattended and automated startup/restart fails.

• **How to verify:** Confirm the admin credential is chosen and stored per your secrets policy, and that a boot-identity approach is agreed (encrypted `boot.properties` under the server's `security` directory).

• **How to implement if missing:** Define the admin credential, plan production mode at creation time, and create the boot-identity file after first start (covered in the Implementation Guide).

7. Administrative tooling: WLST, shell, SQL client

• **Description:** Working **WLST (Jython)** from the WebLogic install, OS **shell** access as the product user, and a **SQL client** (SQL*Plus / SQLcl) — the interfaces this solution's steps and scripts use. (Automation here is WLST/shell/SQL, **not** PowerShell.)

• **Why required:** Domain creation, Node Manager enrollment, data-source creation, and validation

are performed through these tools; the Discovery/Validation/Report/Remediation scripts are WLST and shell.

- **How to verify:** Confirm `java weblogic.WLST` (or `ORACLE_HOME/oracle_common/common/bin/wlst.sh`)

launches, that the product user has shell access to `ORACLE_HOME`/`DOMAIN_HOME`, and that the SQL client connects to the C2M service.

- **How to implement if missing:** Install/verify the WebLogic tooling and a SQL client on the host

and grant the product user the needed access.

8. Change control, backup & a non-production pilot

- **Description:** Per the Global Requirements: a **change-control** approval for any production

domain, verified **backups** (domain config, OUAF product/config, database) before disruptive actions, and a **non-production environment** to build and validate `c2m_domain` on **first**.

- **Why required:** The domain is the foundation the whole C2M stack sits on; a mis-built or

mis-changed domain is high-impact. Every step follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a documented rollback.

- **How to verify:** Confirm a backup/restore path for the domain home and a change window/approval

for any production build or change.

- **How to implement if missing:** Establish backups and change control, and stand up the

non-production environment, before touching production.

*Meeting these — a certified WebLogic 14c installer + JDK, a sized host, the product account and directory layout, reachable ports and database, admin credentials and boot-identity plan, the WLST/shell/SQL toolchain, and change control with a pilot — is the gate to Configure. Run the **Discovery** and **ScopeHelper** scripts to capture the environment and pin targets before you build the domain.*

Global Requirements — Oracle Utilities C2M / WebLogic Solutions

These requirements apply to **every** solution in the **Oracle** category (Oracle Utilities Customer to Meter and its Oracle WebLogic Server middleware). Verify each before starting a solution's steps.

> ***Vendor & version boundary (applies to every Oracle solution).*** *Oracle Utilities **Customer to Meter (C2M)**, **Customer Care & Billing (CC&B)**, **Meter Data Management (MDM)**, and **Oracle WebLogic Server** are authored from Oracle's official product documentation and the Oracle Utilities Application Framework (OUAF) install/admin guides. **Version-specific details** (C2M 25.4 / minimum v2.9, WebLogic 14c) are **cited or flagged "verify for your version"** — never invented. Some Oracle documentation and patches are entitlement-gated on **My Oracle Support (MOS)**; where a step requires a MOS note, patch number, or download, the solution points to it rather than reproducing gated content. All work assumes a **non-production environment first** and change control for production.*

1. Licensed Oracle Utilities C2M + WebLogic entitlement

• **Description:** A licensed Oracle Utilities **Customer to Meter** (and the underlying OUAF) plus the supported **Oracle WebLogic Server 14c** and database (Oracle Database) at the versions in scope.

• **Why required:** Installation media, patches, and support all flow from the entitlement.

• **How to verify:** Confirm the license and access to **My Oracle Support**; check the certified version matrix for C2M ↔ WebLogic ↔ Database ↔ Java.

• **How to implement if missing:** Obtain licensing/entitlement and MOS access before proceeding.

2. Certified platform stack (OS, Java, Database, WebLogic)

• **Description:** A host OS, JDK, Oracle Database, and WebLogic version combination **certified** for the target C2M release (per Oracle's certification/compatibility matrix).

• **Why required:** C2M/OUAF only supports certified combinations; uncertified stacks cause install and patch failures and lose supportability.

• **How to verify:** Compare the environment against the C2M release's certification matrix (MOS).

• **How to implement if missing:** Align OS/JDK/DB/WebLogic to a certified combination first.

3. Environment topology (Dev / Test / Prod) & access

• **Description:** Defined **Development, Test, and Production** environments (and any QA/UAT), with OS accounts (e.g. the OUAF/`splunk`-style product user), WebLogic admin credentials, and DB schema access appropriate to the task.

• **Why required:** C2M is deployed and promoted across environments; each solution assumes the target environment exists and you can reach WebLogic, the file system, and the database.

• **How to verify:** Confirm you can log in to the OS product account, the WebLogic Administration Console / WLST, and the C2M database schema.

• **How to implement if missing:** Stand up / obtain access to the environment before starting.

4. Administrative access & tooling (WLST, shell, SQL)

• **Description:** Administrative access and the standard toolchain: **WLST (Jython)** for WebLogic, the OS **shell** (bash/ksh) for OUAF utilities (e.g. `splmanager`, `configureEnv`, `initialSetup`, batch/threadpool scripts), and **SQL*Plus / SQLcl** for the database.

• **Why required:** C2M/WebLogic administration is performed through these interfaces, not a GUI-only workflow; the solutions' automation is authored as WLST/shell/SQL.

• **How to verify:** Confirm WLST connects to the domain, shell access to the product home, and SQL

connectivity to the schema.

- **How to implement if missing:** Provision the access and install the client tooling.

5. Change control, backup & a non-production pilot

- **Description:** A **change-control** process for production changes, verified **backups** (domain config, OUAF product/config, database) before disruptive actions (install, patch, upgrade, migration, tuning), and a **non-production environment** to validate on first.
- **Why required:** Every solution follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a documented rollback; C2M patches/upgrades and WebLogic changes are high-impact.
- **How to verify:** Confirm a backup/restore path and a change window/approval for production.
- **How to implement if missing:** Establish backups and change control before touching production.

6. Collaborating teams & support model

- **Description:** Engagement with the **infrastructure, DBA, network, application-development, and release/test **teams, and (post go-live) the** hypercare / on-call** support model and monitoring stack (e.g. Dynatrace) the environment uses.
- **Why required:** C2M administration is cross-functional; issues span middleware, database, network, and application layers, and production support runs on a defined escalation and monitoring model.
- **How to verify:** Confirm the named contacts, escalation path, and monitoring tooling.
- **How to implement if missing:** Identify the teams and the support/monitoring model before go-live.

*The deployment model across every Oracle solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **WLST (Jython), shell, and SQL** scripts — not PowerShell.*

Implementation Guide

This guide installs and configures **Oracle WebLogic Server 14c (14.1.1)** as the C2M middleware foundation, in order:

install WebLogic 14c → **create the** `c2m_domain` **domain** → **configure Admin / managed servers + Node Manager + machine** → **create the** `C2MDataSource` **data source** → **tune JVM / startup** → **validate.**

It follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with explicit **gates**. Do the whole sequence on a **non-production environment first**; promote to production only under change control with a verified backup. Every step names the same objects defined in the Solution Overview (`c2m_domain`, `AdminServer`, `c2m_ms1`/`c2m_ms2`, `c2m_cluster`, `c2m_machine1`, `C2MDataSource`) — adopt them or map to your convention **once** and reuse consistently.

Commands use placeholders **without angle brackets** (e.g. `your-oracle-home`, `your-db-host`, `your-service`). Replace them with your values; do not leave placeholders in a built domain. Where a

value is release-dependent it is flagged **verify for your version**.

> ***Two ways to do each configuration step.*** *WebLogic gives you the* ***Administration Console*** *(GUI) and* ***WLST*** *(scriptable). This guide shows* ***Admin Console navigation*** *and the equivalent* ***WLST*** *so the build is repeatable. For a new domain, prefer* ***WLST offline*** *(or the Configuration Wizard) to create it; use the Console/WLST-online for later edits. As always:* ***verify against the Oracle WebLogic 14.1.1 documentation*** *for your exact patch level — MOS-gated notes/patches are pointed to, not reproduced.*

Phase 0 — Assess (gate before you build)

1. **Confirm the requirements are met** (see Requirements Guide): certified WebLogic 14c installer + JDK, sized host, OS product account, planned free ports, reachable C2M database + service, WebLogic admin credential, WLST/shell/SQL tooling, change control + backup.
1. **Capture the environment — run Discovery** (read-only): inventory the host, JDK, any existing `ORACLE_HOME`/domains, free ports, and DB reachability.

```
java weblogic.WLST OWLS14C-Discovery.py    # or: run before install with --pre-install
```

1. **Pin your targets — run ScopeHelper**: record the domain name, server names/ports, machine, Node Manager port, and the data-source coordinates the later scripts will act on.

```
./OWLS14C-ScopeHelper.sh --interactive     # writes owls14c-scope.env
```

> ***Gate 0*** *→* ***Configure:*** *certified stack confirmed, host sized, ports free, database reachable, backup/rollback path in place. If any fails,* ***stop*** *and remediate — do not build on a shaky base.*

Phase 1 — Configure

Step 1 — Install WebLogic Server 14c

As the **OS product account**, with `JAVA_HOME` set to the certified JDK:

1. **Launch the installer** into your chosen `ORACLE_HOME` (outside any future domain home):

```
$JAVA_HOME/bin/java -jar fmw_14.1.1.0.0_wls_generic.jar
```

(Installer filename is **verify for your version** — use the file your entitlement provides.)

1. In the installer: set **Installation Location** = `your-oracle-home`, choose the install type the C2M release requires (WebLogic Server or Fusion Middleware Infrastructure — **verify for your version**), let it run prerequisite checks, and complete.
1. **Silent / repeatable install** (recommended for Test/Prod parity) — generate and reuse a response file:

```
$JAVA_HOME/bin/java -jar fmw_14.1.1.0.0_wls_generic.jar -silent \
    -responseFile your-oracle-home/install.rsp -invPtrLoc your-oracle-home/oraInst.loc
```

1. **Verify the install:** `ORACLE_HOME` populated, and:

```
$ORACLE_HOME/oracle_common/common/bin/wlst.sh    # WLST launches
```

> ***Do not** create the domain into `ORACLE_HOME`. Keep `DOMAIN_HOME` separate so patching/reinstall of the binaries never disturbs `c2m_domain`.*

Step 2 — Create the `c2m_domain` domain

Pick **one** method; both produce the same domain.

Option A — Configuration Wizard (GUI):

```
$ORACLE_HOME/oracle_common/common/bin/config.sh
```

Then: **Create a New Domain** → Domain Location = `your-domain-home/c2m_domain` → select the **Basic WebLogic Server Domain** template (add the Coherence Cluster extension if your C2M topology uses Coherence — see the Coherence/JMS/IWS solution) → set the **admin** username/password → **Domain Mode = Production** and the **certified JDK** → (Advanced) select **Administration Server, Node Manager, Topology**. Continue into Step 3 for servers/Node Manager/machine, then **Create**.

Option B — WLST offline (scriptable, preferred for repeatability):

```
# create_c2m_domain.py  — run: java weblogic.WLST create_c2m_domain.py
readTemplate('your-oracle-home/wlserver/common/templates/wls/wls.jar')
cd('/Servers/AdminServer')
set('ListenAddress', 'your-admin-host')      # a real host/IP, not localhost
set('ListenPort', 7001)
cd('/Security/base_domain/User/weblogic')    # set admin credentials
cmo.setName('your-admin-user')
cmo.setPassword('your-admin-password')
setOption('ServerStartMode', 'prod')         # production mode
setOption('OverwriteDomain', 'true')
writeDomain('your-domain-home/c2m_domain')
closeTemplate()
exit()
```

Verify: `DOMAIN_HOME` (`your-domain-home/c2m_domain`) exists with `bin/`, `config/config.xml`, and `nodemanager/` populated.

Step 3 — Configure Admin Server, managed server(s), machine

If you used the Wizard you can do this inline; either way, here is the WLST-online form (start `AdminServer` first — Step 5 — then connect and edit). **Admin Console** path is given for each.

1. **Managed server** `c2m_ms1` — *Console:* **Environment** → **Servers** → **New** → Name `c2m_ms1`,

Listen Address = `your-ms1-host`, Listen Port = your planned port, **not** part of a cluster yet (or assign to `c2m_cluster`). *WLST:*

```
edit(); startEdit()
cmo.createServer('c2m_ms1')
cd('/Servers/c2m_ms1')
cmo.setListenAddress('your-ms1-host'); cmo.setListenPort(7003)
```

1. **(HA) Cluster** `c2m_cluster` **+ second server** `c2m_ms2` — *Console:* **Environment →

Clusters → New **(**`c2m_cluster`**), then create** `c2m_ms2` **and set each server's** Cluster** to `c2m_cluster`. *WLST:*

```
cmo.createCluster('c2m_cluster')
cmo.createServer('c2m_ms2')
cd('/Servers/c2m_ms2'); cmo.setListenAddress('your-ms2-host'); cmo.setListenPort(7005)
cd('/Servers/c2m_ms1'); cmo.setCluster(getMBean('/Clusters/c2m_cluster'))
cd('/Servers/c2m_ms2'); cmo.setCluster(getMBean('/Clusters/c2m_cluster'))
```

1. **Machine** `c2m_machine1` — *Console:* **Environment** → **Machines** → **New** (Unix Machine),

then **Node Manager** tab → Type SSL (or Plain), Listen Address = **specific host/IP, not** localhost, Listen Port = your Node Manager port. Assign servers under the machine's **Servers** tab. *WLST:*

```
cmo.createUnixMachine('c2m_machine1')
cd('/Machines/c2m_machine1/NodeManager/c2m_machine1')
cmo.setListenAddress('your-nm-host'); cmo.setListenPort(5556); cmo.setNMType('SSL')
cd('/Servers/AdminServer'); cmo.setMachine(getMBean('/Machines/c2m_machine1'))
cd('/Servers/c2m_ms1');     cmo.setMachine(getMBean('/Machines/c2m_machine1'))
# (c2m_ms2 likewise for HA)
save(); activate()
```

> ***Keep the C2M application off*** AdminServer***.*** *The C2M/OUAF EAR (*SPLWeb/SPLService*) is deployed to the* ***managed server(s)*** *— deploying it in a separate solution. Here we only build the domain that receives it.*

Step 4 — Configure the per-domain Node Manager

The **per-domain** Node Manager config already lives in DOMAIN_HOME/nodemanager after domain creation. Finalize it and enroll the domain:

1. **Set Node Manager properties** in DOMAIN_HOME/nodemanager/nodemanager.properties:

ListenAddress=your-nm-host (not localhost), ListenPort=5556, SecureListener=true (SSL) or false (Plain, non-production only), and — so Node Manager can start managed servers itself — StartScriptEnabled=true.

1. **Enroll the domain with Node Manager** (from WLST **online**, connected to AdminServer) so it

trusts and can manage the servers:

```
nmEnroll('your-domain-home/c2m_domain', 'your-domain-home/c2m_domain/nodemanager')
```

1. **Node Manager type — per-domain vs per-host:** this solution uses **per-domain** (self-contained

under DOMAIN_HOME), the modern default. Choose **per-host** only if a site standard requires one Node Manager serving multiple domains on the host; if so, configure it under ORACLE_HOME and register each domain — see Oracle's *Administering Node Manager* (14.1.1) documentation.

Step 5 — Start Node Manager and the Administration Server

From DOMAIN_HOME/bin, as the product user:

1. **Node Manager:**

```
nohup ./startNodeManager.sh > $DOMAIN_HOME/nm.out 2>&1 &
```

1. **Administration Server:**

```
nohup ./startWebLogic.sh > $DOMAIN_HOME/admin.out 2>&1 &
```

In production mode you are prompted for the admin credential on first start. Then create a **boot-identity file** so it (and the managed servers) start unattended under Node Manager — create DOMAIN_HOME/servers/AdminServer/security/boot.properties with username= / password= (WebLogic encrypts it on next start); repeat per managed server directory.

1. **Reach the Console:** http://your-admin-host:7001/console — log in with the admin credential.

Step 6 — Create the C2MDataSource *JDBC data source*

Point WebLogic at the C2M database. Choose **generic** (single-instance DB) or **Active GridLink** (Oracle **RAC**).

Console: **Services** → **Data Sources** → **New** → **Generic Data Source** (or **GridLink**). Name = `C2MDataSource`, JNDI Name = `jdbc/C2MDataSource`, Database Type = **Oracle**. Driver: the Oracle Thin driver (XA or non-XA per your transaction needs). Enter the connection: host = `your-db-host`, port = `your-db-port`, **service name** = `your-service` (use service, not SID, for RAC). Set the schema credentials. **Target** the data source at `c2m_ms1` (and `c2m_cluster` for HA) — **not** `AdminServer`. On the connection-pool tab set **Test Connections On Reserve = true** with **Test Table Name =** `SQL ISVALID` (or `SQL SELECT 1 FROM DUAL`), and size **Initial/Max Capacity** for your load (**verify for your version** against C2M sizing).

WLST (generic, online):

```
edit(); startEdit()
cmo.createJDBCSystemResource('C2MDataSource')
cd('/JDBCSystemResources/C2MDataSource/JDBCResource/C2MDataSource')
cmo.setName('C2MDataSource')
cd('.../JDBCDataSourceParams/C2MDataSource'); set('JNDINames', jarray.array([String('jdbc/C2MDataSource')]
cd('.../JDBCDriverParams/C2MDataSource')
cmo.setUrl('jdbc:oracle:thin:@//your-db-host:your-db-port/your-service')
cmo.setDriverName('oracle.jdbc.OracleDriver')
cmo.setPasswordEncrypted('your-schema-password')
cd('.../Properties/C2MDataSource'); cmo.createProperty('user'); cd('Properties/user'); cmo.setValue('your-
cd('.../JDBCConnectionPoolParams/C2MDataSource')
cmo.setTestConnectionsOnReserve(true); cmo.setTestTableName('SQL ISVALID')
cmo.setInitialCapacity(5); cmo.setMaxCapacity(30)
cd('/SystemResources/C2MDataSource'); set('Targets', jarray.array([ObjectName('com.bea:Name=c2m_ms1,Type=S
save(); activate()
```

(For **GridLink/RAC**, use the RAC SCAN/service URL and enable ONS/FAN per Oracle's *Administering JDBC Data Sources* 14.1.1.) **Verify the target list** does not include `AdminServer`.

Step 7 — Tune JVM / startup

C2M managed-server JVMs are memory- and GC-sensitive; set this deliberately rather than accepting template defaults.

1. **Heap / GC** — set `USER_MEM_ARGS` for the managed server(s) (do not hand-edit `startWebLogic.sh`;

override in `setUserOverrides.sh` / `setUserOverridesLate.sh` under `DOMAIN_HOME/bin`, which `setDomainEnv.sh` sources):

```
# setUserOverrides.sh  (example — size per your host & C2M sizing guidance: verify for your version)
USER_MEM_ARGS="-Xms4g -Xmx4g -XX:MetaspaceSize=512m -XX:MaxMetaspaceSize=512m"
export USER_MEM_ARGS
```

1. **Production mode** — confirm the domain is in production mode (set at creation). Confirm

`StartScriptEnabled=true` so Node Manager applies these startup args when it launches servers.

1. **Boot identity** — confirm `boot.properties` exists for each server (Step 5) so Node Manager

starts them unattended.

1. **Start managed servers** — via Node Manager (Console: **Environment → Servers → Control →

Start**, or WLST `nmStart('c2m_ms1')`), or directly for a quick pilot:

```
nohup ./startManagedWebLogic.sh c2m_ms1 t3://your-admin-host:7001 > $DOMAIN_HOME/ms1.out 2>&1 &
```

> ***Gate 1*** → ***Pilot:*** *WebLogic installed,* `c2m_domain` *created, Admin + managed server(s) + machine configured, per-domain Node Manager enrolled,* `C2MDataSource` *created with Test-On-Reserve, JVM args set. No template placeholders remain. Configuration validates in WLST/Console without errors.*

Phase 2 — Pilot (one non-production node)

Bring the domain up on **one** environment/node and confirm the baseline:

- **Node Manager** running and reachable from the Admin Console/WLST.
- `AdminServer` and `c2m_ms1` both in state `RUNNING`.
- `C2MDataSource` targeted at `c2m_ms1`, **Test = passed** (Console **Monitoring** → **Testing**, or WLST test).
- **JVM args** as intended on `c2m_ms1` (heap/metaspace reflect `USER_MEM_ARGS`).

> ***Gate 2*** → ***Validate:*** *a clean* ***single-node*** *domain — servers* `RUNNING`*, data source reserving connections, Node Manager in control —* ***before*** *any horizontal scale.*

Phase 3 — Validate (evidence, not assumption)

Run the scripts and confirm against the Testing Guide:

```
java weblogic.WLST OWLS14C-Validation.py     # servers RUNNING, DS test, NM reachable, JVM args
java weblogic.WLST OWLS14C-Report.py         # health/coverage summary
```

- **Validation** must report: servers `RUNNING`, `C2MDataSource` test **pass**, Node Manager **reachable**, JVM args **as configured**, no `config.xml` placeholders, machine not on `localhost`.
- If Validation flags a fixable drift (e.g. Test-On-Reserve off, data source on `AdminServer`, `StartScriptEnabled=false`), preview and apply with **Remediation** — **dry-run first**:

```
./OWLS14C-Remediation.sh                # DRY-RUN: prints what it would change
./OWLS14C-Remediation.sh --apply        # apply, after review + backup
```

> ***Gate 3*** → ***Expand:*** *Validation is green with evidence (Report attached to the change record).*

Phase 4 — Expand

Scale and promote under change control, **re-validating** at each step:

1. **Add the second managed server / cluster** (`c2m_ms2` in `c2m_cluster`), target `C2MDataSource` at the cluster, start via Node Manager, re-run Validation.
1. **Add machines / nodes** for additional hosts (each with its own Node Manager and machine definition), keeping to a **specific IP/hostname** per machine.
1. **Promote the pattern across environments** (Dev → Test → Prod) using the same WLST scripts and `owls14c-scope.env` values per environment, each behind its own change approval and backup.

1. **Hand off to the layers above** — the domain is now ready to **receive the C2M/OUAF deployment** (`SPLWeb`/`SPLService`), Coherence/JMS/IWS, batch threadpools, SSO/SSL, and monitoring, each covered by its own solution.

Rollback (any phase)

- **Config change (data source, server, JVM):** WebLogic keeps `config.xml` backups (`config/config.xml.booted` / pending edits). Revert the edit session or restore the domain config from your pre-change backup, then restart the affected server.
- **Domain build gone wrong:** because `DOMAIN_HOME` is **outside** `ORACLE_HOME`, remove/recreate the domain without touching the binaries; restore from the domain backup if needed.
- Always take a **verified backup of the domain home** (and note the DB/OUAF state) before a production change; record the rollback step in the change record.

Testing Guide

Testing proves the WebLogic 14c foundation for C2M is actually sound — **domain up, managed server(s) running, Node Manager in control, and data sources healthy** — with **evidence**, not assumption. Run these tests at the **Pilot** gate (one node) and again after each **Expand** step.

Each test states what to check, **how** (Admin Console navigation and/or WLST, plus the solution's scripts), and the **pass criteria**. Run the read-only **Validation** and **Report** scripts to collect most of this automatically; the manual checks below explain what those scripts assert.

Use placeholders **without angle brackets** (`your-admin-host`, `your-service`) — substitute your values.

1. Installation & domain structure

- **Check:** WebLogic 14c installed in `ORACLE_HOME`, and `c2m_domain` created **outside** it with a populated `bin/`, `config/config.xml`, and `nodemanager/`.
- **How:** on the host, confirm the directory trees; `cat $DOMAIN_HOME/config/config.xml | head` shows the domain; WLST launches (`java weblogic.WLST`). Or run `OWLS14C-Discovery.py`.
- **Pass:** `ORACLE_HOME` and a separate `DOMAIN_HOME` both present; `config.xml` names `c2m_domain`; **no template placeholders** left in `config.xml` (e.g. no leftover `localhost` machine listen address).

2. Node Manager running and reachable

- **Check:** the per-domain Node Manager is up and the Admin Console/WLST can reach it to control servers.

• **How:** *WLST:* `nmConnect('your-nm-user','your-nm-pass','your-nm-host',5556,'c2m_domain',

'your-domain-home/c2m_domain') then nm() returns true. *Console:* **Environment** → **Machines** → **c2m_machine1** → **Monitoring** → **Node Manager Status** = **Reachable**. Or OWLS14C-Validation.py`.

• **Pass:** Node Manager status **Reachable**; `nmConnect` succeeds; the machine listen address is a

specific host/IP, not `localhost`.

3. Administration Server RUNNING

• **Check:** `AdminServer` is up in production mode and the Console is reachable.

• **How:** browse `http://your-admin-host:7001/console` and log in; *WLST:* `connect(...)` then

`state('AdminServer')`. *Console:* **Environment** → **Servers** shows `AdminServer` state **RUNNING**.

• **Pass:** `AdminServer` state = `RUNNING`, Console reachable, health **OK**.

4. Managed server(s) RUNNING

• **Check:** `c2m_ms1` (and `c2m_ms2` for HA) start under Node Manager and reach `RUNNING`.

• **How:** *Console:* **Environment** → **Servers** → **Control** → start → state goes **RUNNING**; *WLST:*

`nmStart('c2m_ms1')` then `state('c2m_ms1')`. Confirm no SEV/ERROR at startup in `DOMAIN_HOME/servers/c2m_ms1/logs/c2m_ms1.log`. Or `OWLS14C-Validation.py`.

• **Pass:** each managed server = `RUNNING`, health **OK**, started **via Node Manager**

(not just a foreground `startManagedWebLogic.sh`), and the C2M app is **not** deployed to `AdminServer`.

5. Data source healthy (Test-On-Reserve)

• **Check:** `C2MDataSource` connects to the C2M database and passes its reserve-time health test,

and is targeted at the **managed server(s)**, not `AdminServer`.

• **How:** *Console:* **Services** → **Data Sources** → **C2MDataSource** → **Monitoring** → **Testing** → select

`c2m_ms1` → **Test Data Source** = **passed**; check **Targets** = `c2m_ms1`/`c2m_cluster`. *WLST:* invoke `testPool` on the runtime MBean, or confirm connections reserve without error. Or `OWLS14C-Validation.py`.

• **Pass:** data-source test **passed** on each target; **Test Connections On Reserve = true** with a

valid **Test Table Name**; pool **Initial/Max Capacity** set (not left at defaults blindly); target list **excludes** `AdminServer`.

6. Database round-trip from the host (independent of WebLogic)

• **Check:** the same DB coordinates the data source uses are reachable and log in from the host —

isolates "DB/network problem" from "WebLogic config problem."

• **How:** `sqlplus your-schema-user@//your-db-host:your-db-port/your-service` (or `tnsping`); a

SQL check script (`OWLS14C-*.sql`) can confirm the schema is present.

• **Pass:** login succeeds against the **service name** (not SID for RAC); the C2M schema objects

are visible. If this fails but you expected the data source to work, the issue is DB/network, not WebLogic.

7. JVM / startup configuration in effect

• **Check:** the managed server(s) actually run with the intended heap/metaspace and production

mode — i.e. `USER_MEM_ARGS` took effect via Node Manager (`StartScriptEnabled=true`).

• **How:** *Console:* **Environment** → **Servers** → **c2m_ms1** → **Monitoring** → **Performance** shows heap

sizes; or inspect the server's runtime args (`ps` on the JVM, or WLST JVMRuntime). Confirm `boot.properties` exists so startup is unattended.

• **Pass:** running heap/metaspace **match** the configured `USER_MEM_ARGS`; domain in **production

mode**; server starts** unattended** under Node Manager (no credential prompt), auto-restart enabled.

8. Coverage / regression — the Report

• **Check:** a single health/coverage snapshot of the whole foundation to attach to the change

record and to re-run after each Expand step (regression baseline).

• **How:** `java weblogic.WLST OWLS14C-Report.py` — summarizes servers & states, Node Manager

reachability, data-source targets & test results, JVM args, and placeholder/`localhost` findings.

• **Pass:** Report shows **all** servers `RUNNING`, Node Manager reachable, `C2MDataSource` healthy

and correctly targeted, JVM args as configured, and **zero** placeholder/`localhost`/AdminServer- targeting findings.

Test summary gate

#	Test	Pass criteria
1	Install & domain structure	`ORACLE_HOME` + separate `DOMAIN_HOME`; no placeholders in `config.xml`
2	Node Manager	**Reachable**; machine on real host/IP, not `localhost`
3	Admin Server	`AdminServer` = **RUNNING**, Console reachable
4	Managed server(s)	`c2m_ms1`(/`c2m_ms2`) = **RUNNING** via Node Manager; app not on Admin

#	Test	Pass criteria
5	Data source	`C2MDataSource` test **passed**; Test-On-Reserve on; not targeted at Admin
6	DB round-trip	SQL login to the **service** succeeds from host
7	JVM / startup	Heap/metaspace match config; production mode; unattended start
8	Report	All green; zero placeholder/localhost/Admin-targeting findings

All eight pass → the domain has cleared the Validate gate. A failure at any test sends you to the Troubleshooting Guide; fixable config drift can be previewed and applied with the Remediation script (dry-run first). Re-run this suite after every **Expand** change.

Troubleshooting Guide

The common failures when installing and configuring WebLogic 14c for C2M, organized by **symptom** → **likely cause** → **resolution**. Start with the read-only **Discovery** and **Validation** scripts — they surface most of these mechanically. Fixable config drift can be previewed and applied with the **Remediation** script (**dry-run first**, backup before `--apply`).

Placeholders are **without angle brackets** (`your-domain-home`, `your-db-host`, `your-service`) — substitute your values. Version-specific behavior is **verify for your version** against Oracle's WebLogic 14.1.1 documentation; MOS-gated notes/patches are pointed to, not reproduced.

Install & domain creation

Installer fails or WLST won't launch after install

- **Likely cause:** wrong/uncertified JDK, `JAVA_HOME` not set, insufficient disk, or wrong

installer type for the C2M release.

- **Resolution:** confirm `java -version` is the **certified** JDK and `JAVA_HOME` points at it;

ensure free disk for `ORACLE_HOME`; re-run the correct installer (**verify for your version**). Verify with `$ORACLE_HOME/oracle_common/common/bin/wlst.sh`.

`config.sh` / *WLST domain creation errors, or domain half-created*

- **Likely cause:** domain home path not writable by the product user, path **inside** `ORACLE_HOME`,

or a template mismatch.

- **Resolution:** create `DOMAIN_HOME` **outside** `ORACLE_HOME`, owned by the product user; re-run

creation with `setOption('OverwriteDomain','true')` (WLST) after clearing the partial domain. Confirm the selected template matches your topology (add Coherence extension only if used).

`config.xml` *still contains* `localhost` *or placeholder listen addresses*

- **Likely cause:** accepted defaults in the Wizard, or WLST script left placeholders.
- **Resolution:** set **specific host/IP** listen addresses for `AdminServer`, managed servers, and

the machine/Node Manager. `localhost` breaks Node Manager control and clustering. Validation flags this; Remediation can correct the machine address.

Administration Server

AdminServer won't start / prompts endlessly for credentials

- **Likely cause:** production mode with **no boot-identity file**, or a wrong/forgotten admin

credential.

- **Resolution:** create `DOMAIN_HOME/servers/AdminServer/security/boot.properties` with

`username=` / `password=` (WebLogic encrypts on next start). If the credential is lost, reset it via the offline security tooling (see Oracle docs). Check `admin.out` / `servers/AdminServer/logs/AdminServer.log` for the actual error.

AdminServer starts but Console is unreachable

- **Likely cause:** listen port in use / firewalled, or bound to the wrong address.
- **Resolution:** confirm the listen port is free (`ss -tlnp`) and open in the firewall; confirm the

listen address is the host you're browsing to; try `http://your-admin-host:7001/console`.

Node Manager

Node Manager "not reachable" from Console/WLST

- **Likely cause:** Node Manager not running, listening on `localhost`, port mismatch, or the domain

never enrolled.

- **Resolution:** start it (`DOMAIN_HOME/bin/startNodeManager.sh`); set

`ListenAddress=your-nm-host` (not `localhost`) and the correct `ListenPort` in `nodemanager.properties`; enroll the domain: WLST `nmEnroll('your-domain-home/c2m_domain', 'your-domain-home/c2m_domain/nodemanager')`. Test with `nmConnect(...)`.

Managed server starts in foreground but not via Node Manager

- **Likely cause:** `StartScriptEnabled=false`, so Node Manager doesn't run the start script (and

your JVM args don't apply either).

- **Resolution:** set `StartScriptEnabled=true` in `nodemanager.properties`, restart Node Manager,

and start the server via `nmStart('c2m_ms1')` / Console **Control** → **Start**. Remediation can set this flag.

Node Manager SSL / trust errors

• **Likely cause:** `SecureListener=true` without matching trust between the domain and Node Manager, or a stale enrollment.

• **Resolution:** re-run `nmEnroll` to re-establish trust; confirm the machine's Node Manager type (SSL vs Plain) matches `nodemanager.properties`. (Full keystore/SSL hardening is a separate solution.)

Managed servers

Managed server won't reach RUNNING (stuck in STARTING/ADMIN/FAILED)

• **Likely cause:** data-source failure at boot, JVM `OutOfMemoryError`, port conflict, or a deployment error (out of scope here).

• **Resolution:** read `servers/c2m_ms1/logs/c2m_ms1.log`. If the data source is failing, fix it (below). If OOM, raise heap (`USER_MEM_ARGS`). If a port conflict, pick a free port. Confirm the server isn't inadvertently hosting the C2M app *and* admin duties.

C2M app accidentally deployed to AdminServer

• **Likely cause:** wrong deployment target chosen (application deployment is a separate solution, but the mistake surfaces here as admin instability).

• **Resolution:** target the C2M application and its data source at the **managed server(s)/cluster**, never `AdminServer`. Validation flags data sources mis-targeted at `AdminServer`.

Data sources (`C2MDataSource`)

Data-source test fails / connections can't be reserved

• **Likely cause:** wrong URL (SID vs **service name**), bad schema credentials, DB/listener down, network/firewall block, or uncertified driver.

• **Resolution:** confirm the DB round-trip **outside** WebLogic first — `sqlplus your-schema-user@//your-db-host:your-db-port/your-service` or `tnsping`. Then fix the data-source URL to use the **service name**, correct the credentials, and confirm the JDBC driver is certified for the DB release (**verify for your version**).

Intermittent stale-connection errors under load

• **Likely cause: Test Connections On Reserve** disabled, so dead connections are handed out after a DB blip; or pool too small.

• **Resolution:** enable Test-On-Reserve with a valid **Test Table Name** (`SQL ISVALID` / `SQL SELECT 1 FROM DUAL`); raise Initial/Max Capacity per load (**verify for your version**). Remediation can enable Test-On-Reserve.

RAC failover not working

• **Likely cause:** a **generic** data source (or SID-based URL) against a RAC database instead of **Active GridLink** with FAN/ONS.

• **Resolution:** use an Active GridLink data source with the RAC SCAN/service URL and ONS/FAN enabled (see Oracle *Administering JDBC Data Sources* 14.1.1).

JVM / startup / memory

`OutOfMemoryError` *(heap or metaspace) on a managed server*

• **Likely cause:** heap/metaspace left at template defaults, undersized for C2M's memory profile.

• **Resolution:** set `USER_MEM_ARGS` in `DOMAIN_HOME/bin/setUserOverrides.sh` (e.g. pin `-Xms=-Xmx`, size metaspace), ensure `StartScriptEnabled=true` so it applies via Node Manager, restart the server. Right-size per C2M sizing guidance (**verify for your version**).

Tuned JVM args don't take effect

• **Likely cause:** args set only in a hand-edited `startWebLogic.sh` (lost on regeneration), or `StartScriptEnabled=false` so Node Manager ignores the start script.

• **Resolution:** put args in `setUserOverrides.sh` (sourced by `setDomainEnv.sh`), set `StartScriptEnabled=true`, restart via Node Manager, and confirm the running heap in Console **Monitoring** → **Performance**.

Intermittent failures under load, no obvious config cause

• **Likely cause:** OS **ulimits** (open files / processes) too low for the product user.

• **Resolution:** raise `nofile`/process limits for the product user and restart the servers.

When to escalate

• **MOS-gated** issues (a specific patch, certification question, or product bug) — raise a Service Request with My Oracle Support; the exact patch/note is entitlement-gated and pointed to, not reproduced here.

• **Cross-layer** problems (application deployment, Coherence/JMS/IWS, SSO/SSL, batch, DB tuning) — those belong to their own solutions; this guide covers the **domain/middleware foundation** only.

• Always capture the **Report** output and the relevant server / Node Manager logs before escalating, and confirm the change record's backup/rollback path is intact.

Best Practices

Practices for a WebLogic 14c domain that carries C2M well over its life — not just up on day one. They reinforce the **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model and the named topology (`c2m_domain`, `AdminServer`, `c2m_ms1`/`c2m_ms2`, `c2m_cluster`, `c2m_machine1`, `C2MDataSource`).

Where a specific value depends on the release, **verify for your version** against Oracle's documentation and certification matrix.

Domain layout & topology

- **Keep** `DOMAIN_HOME` **outside** `ORACLE_HOME`**.** Binaries and domain have different lifecycles;

separating them lets you patch/reinstall WebLogic without disturbing `c2m_domain`, and back them up independently.

- **Never deploy the C2M application to** `AdminServer`**.** Reserve `AdminServer` for administration;

run C2M (`SPLWeb`/`SPLService`) on the managed server(s). This isolates admin availability from app load and is the supported OUAF pattern.

- **Name objects deliberately and reuse the names everywhere** — domain, servers, cluster, machine,

data source. Consistent names across `config.xml`, WLST scripts, and runbooks prevent the "which server did we mean?" class of outage. Map to your site convention **once**, then hold it.

- **Bind to real hostnames/IPs, never** `localhost`**.** A machine or listen address on `localhost`

breaks Node Manager control and (later) clustering/Coherence. This is a validation check for a reason.

- **Start single-node, then scale.** Prove one managed server clean before adding `c2m_ms2` /

`c2m_cluster`. HA on top of an unproven single node just multiplies the problem.

Node Manager

- **Use the per-domain Node Manager** (under `DOMAIN_HOME/nodemanager`) unless a site standard

mandates per-host. Per-domain keeps each domain self-contained and easy to move/patch.

- **Run servers under Node Manager, not foreground scripts.** Foreground `startManagedWebLogic.sh`

is fine for a quick pilot, but production servers should start/stop/**auto-restart** through Node Manager so a crash recovers itself.

- **Enable** `StartScriptEnabled=true` so Node Manager applies your `setUserOverrides.sh` JVM args

when it launches servers — otherwise your carefully-tuned heap silently doesn't apply.

- **Secure Node Manager** (`SecureListener=true`, SSL) outside throwaway sandboxes, and register

the domain with `nmEnroll` so trust is established once and reused.

- **Install Node Manager as a service/boot job** so it survives host reboots; a rebooted host with

no Node Manager means nothing auto-starts.

Data sources

- **Turn on Test Connections On Reserve** with a lightweight test (`SQL ISVALID` /

`SQL SELECT 1 FROM DUAL`) so WebLogic hands out only live connections — cheap insurance against stale-connection failures after a DB blip.

- **Use the service name, not SID** (required for RAC), and prefer an **Active GridLink** data source when the C2M database is **RAC** so FAN/ONS failover and load balancing work.
- **Target data sources at the managed server(s)/cluster, never** `AdminServer`**.** A data source on `AdminServer` is a classic mis-target that "works" in a demo and fails the moment app load moves to the managed servers.
- **Size the pool for real load, not defaults.** Set Initial/Max Capacity from C2M sizing guidance and observed concurrency (**verify for your version**); watch for pool exhaustion and connection leaks under batch + online concurrency.
- **Keep schema credentials out of scripts.** Set them in the data source (WebLogic encrypts them); never bake passwords into WLST files committed anywhere.

JVM / startup & memory

- **Set heap deliberately via** `setUserOverrides.sh`, not by editing generated startup scripts. Overrides survive domain re-generation; hand-edits to `startWebLogic.sh` do not.
- **Right-size heap and metaspace for C2M** and pin `-Xms` = `-Xmx` in production to avoid heap resize churn; choose/verify the GC per your JDK and C2M sizing guidance (**verify for your version**). C2M is memory-sensitive — undersized heap surfaces as `OutOfMemoryError` under load.
- **Run production domains in production mode** with an encrypted **boot-identity file** so servers start unattended under Node Manager.
- **Raise OS ulimits** (open files, processes) for the product user; low `nofile` limits cause intermittent, hard-to-diagnose failures under load.

Security & operations

- **Least privilege:** dedicated OS product user owns the install/domain; distinct WebLogic admin and Node Manager credentials; rotate per policy. (SSL/keystore and SSO have their own solution — point to it, don't hand-roll.)
- **Back up before every change** — the domain home (config), plus note the OUAF product/config and DB state — and keep the rollback step in the change record. `DOMAIN_HOME` being outside `ORACLE_HOME` makes domain-only restore clean.
- **Keep environments parallel.** Build Dev → Test → Prod from the **same WLST scripts** with a per-environment `owls14c-scope.env`, so the domain that reaches production is the one you validated.
- **Change control for production**, always: certified stack, backup, change window, validation evidence (attach the Report), documented rollback.

Validate continuously

- **Re-run Discovery/Validation/Report after every change** (server added, data source retargeted, JVM tuned, patch applied) so drift is caught by evidence, not by an incident.
- **Preview remediation before applying** — the Remediation script is **dry-run by default**; review its plan, take a backup, then `--apply`. Never let an automated fix touch a production domain unreviewed.
- **Treat the Report as your regression baseline** — a green Report before a change and a green Report after is the simplest proof you didn't regress the foundation.

Deliverables Summary

Everything included in **Oracle WebLogic 14c for C2M — Install, Configure & Domain Setup**, and how the pieces fit together. This solution builds the **WebLogic 14c middleware foundation** that C2M is deployed onto: install WebLogic → create the `c2m_domain` domain → configure Admin / managed servers + Node Manager + machine → create the `C2MDataSource` data source → tune JVM / startup → validate — pilot-first, with explicit gates.

Documentation

Document	What it gives you
Solution Overview	The named domain topology (`c2m_domain`, `AdminServer`, `c2m_ms1`/`c2m_ms2`, `c2m_cluster`, `c2m_machine1`, `C2MDataSource`), how OUAF/C2M sits on WebLogic, the Assess→Configure→Pilot→Validate→Expand model, and scope boundaries.
Requirements Guide	Each prerequisite — certified WebLogic 14c installer + JDK, sized host, OS product account & directory layout, free ports, reachable DB + service, admin credential & boot-identity plan, WLST/shell/SQL tooling, change control — with **verify** / **implement** steps.
Implementation Guide	The exact ordered build: install WebLogic 14c → create `c2m_domain` → configure Admin/managed servers + Node Manager + machine → create `C2MDataSource` → tune JVM/startup → validate, with **Admin Console navigation** and **WLST** side by side, and rollback.
Testing Guide	Eight tests proving domain up, servers `RUNNING`, Node Manager reachable, data sources healthy, JVM args in effect — with pass criteria and a summary gate.

Document	What it gives you
Troubleshooting Guide	Symptom → likely cause → resolution for the common domain, server, Node Manager, data-source, and JVM failures.
Best Practices	Domain layout, Node Manager, data sources, JVM/memory, security/ops, and continuous validation.

Automation scripts

Five scripts, authentic to WebLogic + C2M administration — **WLST (Jython), shell, and SQL, not PowerShell**. Read-only tools are read-only; the only script that can change anything defaults to **dry-run**.

Script	Type	Mode	What it does
OWLS14C-Discovery	WLST / shell	Read-only	Inventories the environment — host, JDK, `ORACLE_HOME`/domains, servers & states, machine/Node Manager, data sources, free ports, DB reachability. Run before and after building the domain.
OWLS14C-Validation	WLST	Read-only	Asserts the pass criteria — servers `RUNNING`, Node Manager reachable, `C2MDataSource` Test-On-Reserve **passes** and not targeted at `AdminServer`, JVM args as configured, no `localhost`/placeholders.
OWLS14C-Report	WLST	Read-only	Produces a health/coverage snapshot of the whole foundation to attach to the change record and use as a regression baseline.
OWLS14C-Remediation	shell / WLST	**Dry-run default**	Previews and (with `--apply`) fixes common, safe config drift — Test-On-Reserve off, data source on `AdminServer`, `StartScriptEnabled=false`, machine on `localhost`. Backup + review before applying.

Script	Type	Mode	What it does
OWLS14C-ScopeHelper	shell	Read-only	Pins the targets the other scripts act on — domain name, server names/ports, machine, Node Manager host/port, data-source coordinates — into `owls14c-scope.env` for consistent, per-environment runs.

Each script carries a native-comment header (`#/--`) stating **PURPOSE**, **HOW TO RUN**, **EXPECTED RESULTS**, and **NOTES**, and uses placeholders **without angle brackets** (`your-domain`, `your-db-host`).

How the pieces work together

1. **ScopeHelper** captures your environment's names and coordinates → `owls14c-scope.env`.
2. **Discovery** inventories the host/domain (before build, and again after).
3. Build the domain per the **Implementation Guide** (install → domain → servers/Node Manager/ machine → data source → JVM/startup).
1. **Validation** + **Report** prove the **Pilot/Validate** gates with evidence.
2. **Remediation** (dry-run → apply) fixes any flagged drift safely.
3. Re-run Validation/Report after each **Expand** step and every future change.

What this solution does not include (pointed to, not reproduced)

• **Deploying the C2M/OUAF application** (`SPLWeb`/`SPLService`) onto the domain, and OUAF `configureEnv`/`initialSetup` — separate C2M installation/deployment solution.

• **Coherence / JMS / IWS** configuration, **batch threadpool** workers, **SSO / SSL / keystore**, and **WebLogic patching/upgrade** — each its own solution.

• **My-Oracle-Support-gated** notes, patches, and the exact certified WebLogic/JDK/DB versions — the solution points to MOS and flags version-specific values as **verify for your version** rather than reproducing gated content.

Outcome

A correctly built **WebLogic 14c foundation for C2M** — `c2m_domain` created, `AdminServer` and the managed server(s) `RUNNING`, the per-domain **Node Manager** in control, `C2MDataSource` healthy and correctly targeted, and JVM / startup configured deliberately — **proven on a pilot before you scale**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OWLS14C-Discovery.py

Purpose: Inventory the WebLogic 14c environment that C2M runs on so you know what you have before and after building the domain. Reports the domain name, servers and their states, cluster, machine and Node Manager, JDBC data sources and their targets, and JVM/production-mode settings. Reads only — it changes nothing. HOW TO RUN Offline (inspect config on disk, no running domain needed): java weblogic.WLST OWLS14C-Discovery.py --domain your-domain-home/c2m_domain Online (also read live server states, needs a running AdminServer): java weblogic.WLST OWLS14C-Discovery.py --domain your-domain-home/c2m_domain \ --admin-url t3://your-admin-host:7001 --user your-admin-user Values also read from owls14c-scope.env (see OWLS14C-ScopeHelper.sh) if present. EXPECTED RESULTS A printed inventory: domain, AdminServer, managed server(s) + states, cluster, machine + Node Manager listen address, data sources + targets, production mode and JVM args. No pass/fail judgment (that is Validation) — this is the raw picture. Exit 0 on success; non-zero only if it cannot read. NOTES White-label, repeatable: run it the same way in Dev/Test/Prod before and after a change to see drift. Strictly read-only — safe against production; it opens no edit session and writes nothing to the domain.

How to run: Offline (inspect config on disk, no running domain needed): java weblogic.WLST OWLS14C-Discovery.py --domain your-domain-home/c2m_domain Online (also read live server states, needs a running AdminServer): java weblogic.WLST OWLS14C-Discovery.py --domain your-domain-home/c2m_domain \ --admin-url t3://your-admin-host:7001 --user your-admin-user Values also read from owls14c-scope.env (see OWLS14C-ScopeHelper.sh) if present. EXPECTED RESULTS A printed inventory: domain, AdminServer, managed server(s) + states, cluster, machine + Node Manager listen address, data sources + targets, production mode and JVM args. No pass/fail judgment (that is Validation) — this is the raw picture. Exit 0 on success; non-zero only if it cannot read. NOTES White-label, repeatable: run it the same way in Dev/Test/Prod before and after a change to see drift. Strictly read-only — safe against production; it opens no edit session and writes nothing to the domain.

OWLS14C-Remediation.sh

Purpose: Correct the common, safe configuration drift the Validation script flags on the WebLogic 14c domain for C2M: Test-On-Reserve OFF on the C2MDataSource data source, the data source targeted at AdminServer instead of the managed servers, and Node Manager StartScriptEnabled=false. DRY-RUN by default — it prints exactly what it would change and touches nothing until you pass --apply. HOW TO RUN Preview (default, no changes): ./OWLS14C-Remediation.sh --domain your-domain-home/c2m_domain \ --admin-url t3://your-admin-host:7001 --user your-admin-user Apply (after review + a verified domain backup): ./OWLS14C-Remediation.sh --apply --domain your-domain-home/c2m_domain \ --admin-url t3://your-admin-host:7001 --user your-admin-user Values also read from owls14c-scope.env if present; password from OWLS14C_ADMIN_PASS. Needs ORACLE_HOME (for WLST) on PATH or exported. EXPECTED RESULTS DRY-RUN: a numbered plan of the fixes it would make, then "No changes made." APPLY: each fix confirmed, the edit session activated, and a note to re-run OWLS14C-Validation.py. Exit 0 on success; non-zero on error. NOTES White-label, repeatable: preview in every environment, apply only after review and backup. Safe by default (dry-run); it only ever adjusts the three low-risk settings above — it never creates/deletes servers or changes credentials. Back up the domain before --apply.

How to run: Preview (default, no changes): ./OWLS14C-Remediation.sh --domain your-domain-home/c2m_domain \ --admin-url t3://your-admin-host:7001 --user your-admin-user Apply (after review + a verified domain backup): ./OWLS14C-Remediation.sh --apply --domain your-domain-home/c2m_domain \ --admin-url t3://your-admin-host:7001 --user your-admin-user Values also read from owls14c-scope.env if present; password from OWLS14C_ADMIN_PASS. Needs ORACLE_HOME (for WLST) on PATH or exported. EXPECTED RESULTS DRY-RUN: a numbered plan of the fixes it would make, then "No changes made." APPLY: each fix confirmed, the edit session activated, and a note to re-run OWLS14C-Validation.py. Exit 0 on success; non-zero on error. NOTES White-label, repeatable: preview in every environment, apply only after review and backup. Safe by default (dry-run); it only ever adjusts the three low-risk settings above — it never creates/deletes servers or changes credentials. Back up the domain before --apply.

OWLS14C-Report.py

Purpose: Produce one health and coverage snapshot of the WebLogic 14c foundation for C2M — servers and states, Node Manager reachability, data sources with their targets and Test-On-Reserve setting, JVM/production mode, and any localhost/AdminServer-targeting findings — to attach to a change record and reuse as a regression baseline. Reads only; changes nothing. HOW TO RUN java weblogic.WLST OWLS14C-Report.py --domain your-domain-home/c2m_domain \ --admin-url t3://your-admin-host:7001 --user your-admin-user \ --out owls14c-report.txt (Values also read from owls14c-scope.env if present. --out is optional; without it the report prints to the screen.) EXPECTED RESULTS A sectioned report (Domain, Servers, Node Manager/Machines, Data Sources, Findings) written to screen and, if --out is given, to a text file. A clean foundation shows all servers RUNNING, Node Manager reachable, data sources healthy and correctly targeted, and zero findings. Exit 0 on success. NOTES White-label, repeatable: capture a report before and after every change so a green-to-green comparison proves you did not regress the foundation. Strictly read-only — opens no edit session; safe against production.

How to run: java weblogic.WLST OWLS14C-Report.py --domain your-domain-home/c2m_domain \ --admin-url t3://your-admin-host:7001 --user your-admin-user \ --out owls14c-report.txt (Values also read from owls14c-scope.env if present. --out is optional; without it the report prints to the screen.) EXPECTED RESULTS A sectioned report (Domain, Servers, Node Manager/Machines, Data Sources, Findings) written to screen and, if --out is given, to a text file. A clean foundation shows all servers RUNNING, Node Manager reachable, data sources healthy and correctly targeted, and zero findings. Exit 0 on success. NOTES White-label, repeatable: capture a report before and after every change so a green-to-green comparison proves you did not regress the foundation. Strictly read-only — opens no edit session; safe against production.

OWLS14C-ScopeHelper.sh

Purpose: Capture, in one place, the environment coordinates the OWLS14C Discovery, Validation, Report and Remediation scripts read: the domain home and name, Admin URL and user, managed server names, machine and Node Manager host/port, and the C2MDataSource data-source name and database service. Writes them to owls14c-scope.env so every script runs against the same, consistent target. HOW TO RUN Interactive (prompts for each value, shows current/default): ./OWLS14C-ScopeHelper.sh --interactive Non-interactive (pass what you know; unset values keep placeholders): ./OWLS14C-ScopeHelper.sh --domain your-domain-home/c2m_domain \ --admin-url t3://your-admin-host:7001 --user your-admin-user Show the current scope file: ./OWLS14C-ScopeHelper.sh --show EXPECTED RESULTS An owls14c-scope.env file in the current directory containing OWLS14C_* variables. The other scripts auto-source it, so you set targets once per environment (Dev/Test/Prod) and reuse them. Exit 0 on

success. NOTES White-label, repeatable: keep one scope file per environment (e.g. owls14c-scope.dev.env) and copy it in as owls14c-scope.env before a run. Read-only against live systems — it never touches the domain or database; it only writes the local scope file. Never store the AdminServer password here — pass it via OWLS14C_ADMIN_PASS or an interactive prompt at run time.

How to run: Interactive (prompts for each value, shows current/default): ./OWLS14C-ScopeHelper.sh --interactive Non-interactive (pass what you know; unset values keep placeholders): ./OWLS14C-ScopeHelper.sh --domain your-domain-home/c2m_domain \ --admin-url t3://your-admin-host:7001 --user your-admin-user Show the current scope file: ./OWLS14C-ScopeHelper.sh --show EXPECTED RESULTS An owls14c-scope.env file in the current directory containing OWLS14C_* variables. The other scripts auto-source it, so you set targets once per environment (Dev/Test/Prod) and reuse them. Exit 0 on success. NOTES White-label, repeatable: keep one scope file per environment (e.g. owls14c-scope.dev.env) and copy it in as owls14c-scope.env before a run. Read-only against live systems — it never touches the domain or database; it only writes the local scope file. Never store the AdminServer password here — pass it via OWLS14C_ADMIN_PASS or an interactive prompt at run time.

OWLS14C-Validation.py

Purpose: Assert the WebLogic 14c foundation for C2M meets its gate criteria: AdminServer and managed server(s) RUNNING, Node Manager reachable, the C2MDataSource data source passes its Test-On-Reserve and is NOT targeted at AdminServer, JVM/production mode as intended, and no localhost/placeholder listen addresses. Reads only — it makes no changes. HOW TO RUN java weblogic.WLST OWLS14C-Validation.py --domain your-domain-home/c2m_domain \ --admin-url t3://your-admin-host:7001 --user your-admin-user (Values also read from owls14c-scope.env if present; password from OWLS14C_ADMIN_PASS or an interactive prompt.) EXPECTED RESULTS One line per check: PASS / FAIL / WARN with a short reason, then a summary. Exit 0 when all checks pass; exit 1 when any check FAILs — so it can gate a pipeline. FAIL points you to the Troubleshooting Guide / Remediation script. NOTES White-label, repeatable: run at the Pilot and Validate gates and after every change to catch drift by evidence, not by incident. Strictly read-only — opens no edit session; safe against production.

How to run: java weblogic.WLST OWLS14C-Validation.py --domain your-domain-home/c2m_domain \ --admin-url t3://your-admin-host:7001 --user your-admin-user (Values also read from owls14c-scope.env if present; password from OWLS14C_ADMIN_PASS or an interactive prompt.) EXPECTED RESULTS One line per check: PASS / FAIL / WARN with a short reason, then a summary. Exit 0 when all checks pass; exit 1 when any check FAILs — so it can gate a pipeline. FAIL points you to the Troubleshooting Guide / Remediation script. NOTES White-label, repeatable: run at the Pilot and Validate gates and after every change to catch drift by evidence, not by incident. Strictly read-only — opens no edit session; safe against production.